



UNGDOMMENS NATURVIDENSKABELIGE FORENING

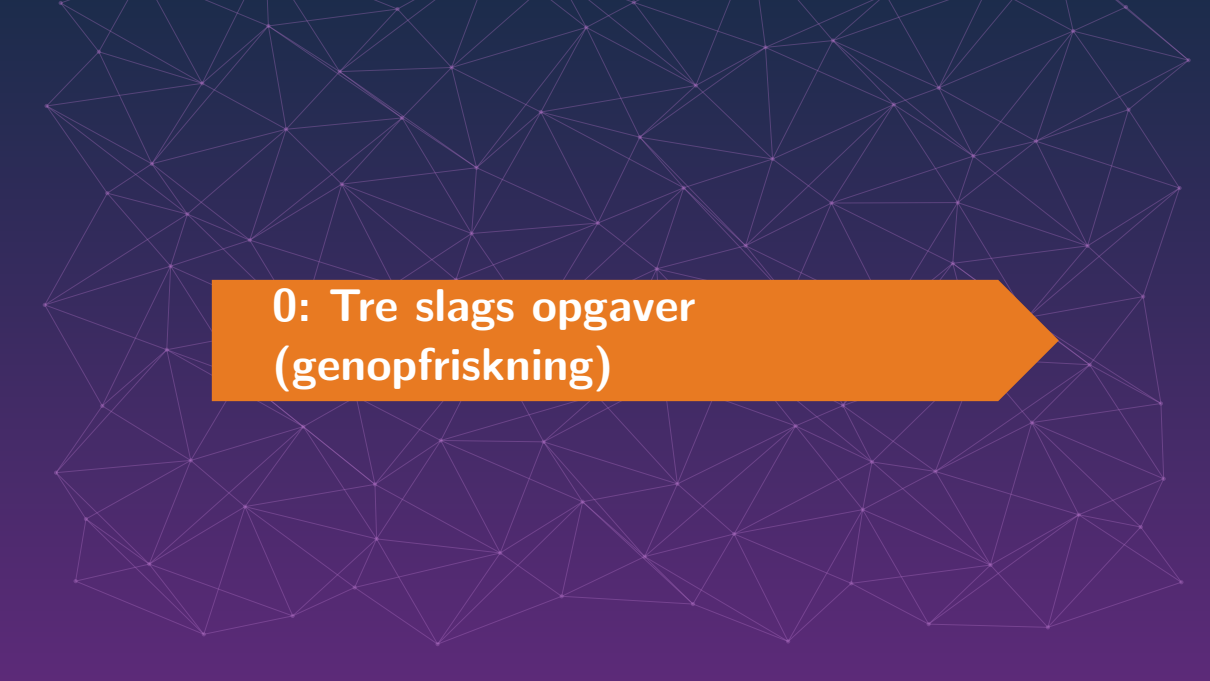
Neurale Netværk og PyTorch Modeller

Vi lærer en computer at kende Mario fra en Goomba



- Kort genopfriskning: hvilke slags opgaver løser vi egentlig?
- **Én neuron** — den mindste byggeklods (og den er bare en linje)
- Fra én neuron til et helt **lag** — og til et **netværk**
- Hvordan et netværk **selv opfinder** nyttige features
- **Live coding:** sådan skriver man en model i PyTorch med klasser

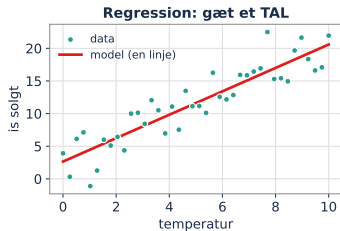
Mål: I forstår hvad der sker inde i kassen — ikke bare at den virker.



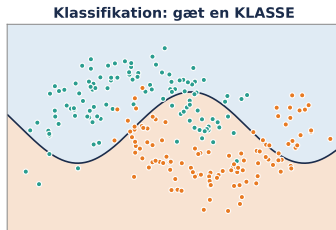
0: Tre slags opgaver (genopfriskning)



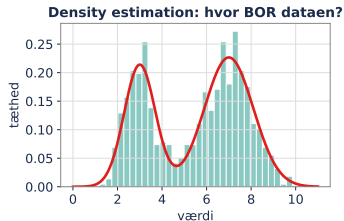
Maskinlæring løser (næsten altid) én af tre opgaver



Regression
gæt et *tal*



Klassifikation
gæt en *klasse*



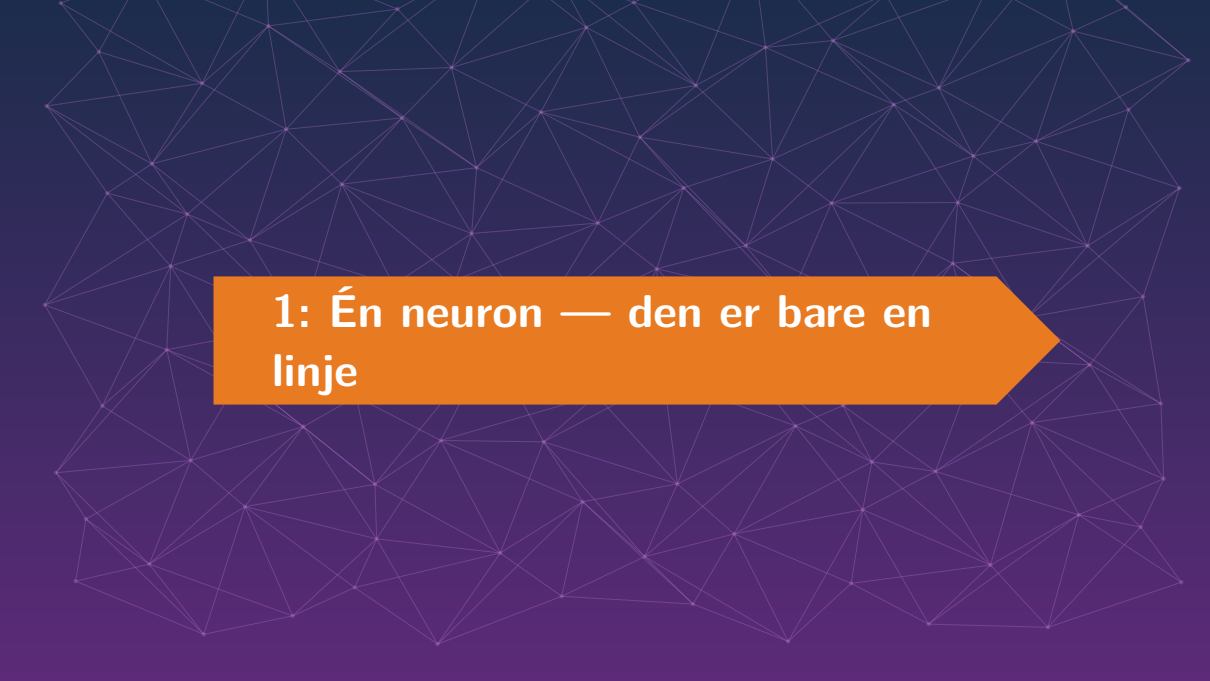
Density estimation
hvor *bor* dataen?

*De fleste af jeres eksempler i dag er **klassifikation** — “hvilken slags er det?”*



Samme netværk kan lave **alle tre** — man skifter bare den sidste lille del ud. Mere om det i morgen. I dag: hvordan bygger man kassen?

- I regressions-dagene byggede I allerede en model: en **linje**.
- En linje er faktisk verdens mindste neurale netværk.
- I dag stabler vi mange af dem oven på hinanden.



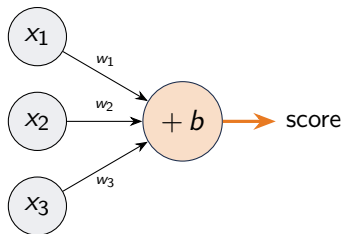
1: Én neuron — den er bare en linje



Hvad gør én neuron?

En neuron får nogle tal ind (*features*) og laver ét tal ud:

- 1 gang hver feature med sin egen **vægt**
- 2 læg det hele sammen
- 3 læg en **bias** til



Matematik-varsel: Skrevet helt ud er det bare gange-og-læggesammen:

$$\text{score} = w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_3 + b$$

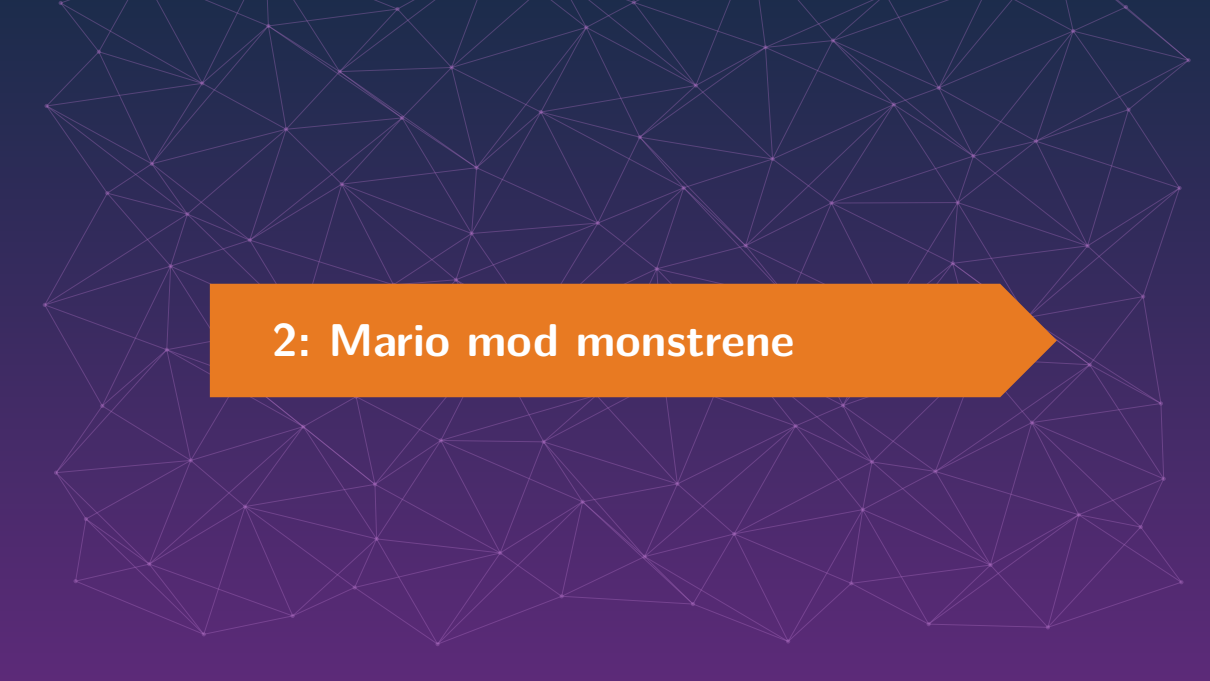
Matematikere skriver den korte version med et sum-tegn: $\text{score} = \sum_i w_i x_i + b$. Det store Σ betyder *præcis* det samme — “læg alle leddene sammen”. Så lad dig ikke skræmme.



- Med én feature bliver det: $\text{score} = w \cdot x + b$.
- Det er jo bare $y = a \cdot x + b$ — en helt almindelig linje!
- I PyTorch hedder en neuron `nn.Linear`. Jeres regressionslinje *var* et neuralt netværk med præcis én neuron.

En neuron = en linje med plads til flere variable. That's it.

Så I byggede et neuralt netværk i går uden at vide det. Læg det på LinkedIn.



2: Mario mod monstrene



Opgaven: hvilken figur er det?

Vi vil kende figuren ud fra dens **træk**. I dag holder vi os til **fire** figurer:



Goomba

lav · ingen arme



Koopa

grøn · oprejst



Bowser

kæmpe · arme+ben



Mario

menneske · rød

4 klasser. Input = træk. Output = "hvem er det?"



En figur er en “opskrift” af features

Netværket arbejder med nogle få **fundne features**. Hver figur er en bestemt blanding:

Figur	Opskrift (fundne features)
Koopa	farve + menneske-lignende
Goomba	proportioner + farve (IKKE menneske-lignende)
Bowser	stor + menneske-lignende + farve
Mario	farve + menneske-lignende + proportioner

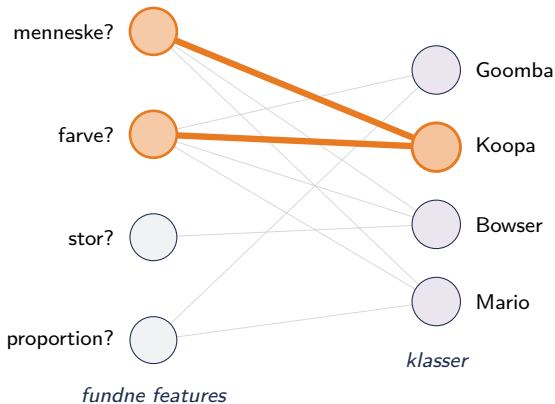
Den figur hvis opskrift passer bedst = højeste score = gættet (softmax i morgen).



Kør det igennem: Koopa



Koopa



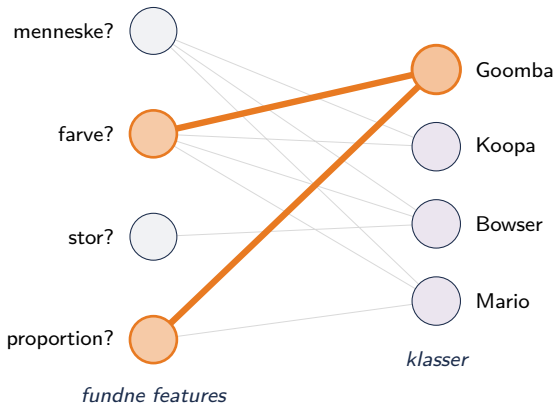
$Koopa = \text{farve} + \text{menneske-lignende} \rightarrow \text{de to features tændt} \rightarrow Koopa.$



...og Goomba



Goomba



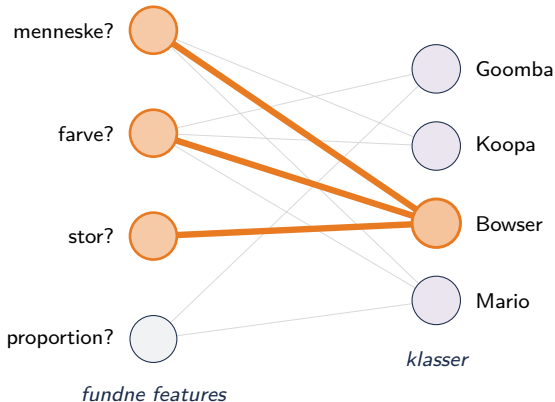
Goomba = proportioner + farve — og IKKE menneske-lignende → Goomba.



...Bowser



Bowser



$Bowser = \textit{stor} + \textit{menneske-lignende} + \textit{farve} \rightarrow Bowser.$

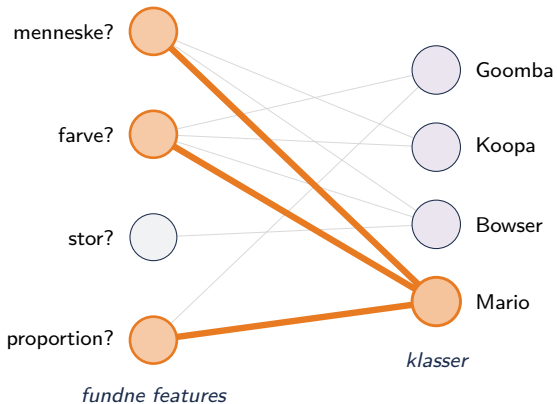
Tre features tændt på én gang. Bowser er ret meget “det hele, men stort”.



...og Mario



Mario



Mario = farve + menneske-lignende + proportioner → Mario.



Hvordan finder den de rigtige vægte? Ét skridt ad gangen

- 1 Start med **tilfældige** vægte (modellen aner intet).
- 2 Vis den en figur, se hvor forkert den gætter (*tabet*).
- 3 Skru en anelse på hver vægt, så gættet bliver lidt bedre.
- 4 Gentag ... tusindvis af gange.

runde 1: "Bowser" (om en Goomba) → runde 50: "måske Goomba?" → runde 500:
"Goomba!" ✓

*Det hedder **træning** — og det er den samme rytme hver gang.*

Vi holder øje med tabet undervejs — og vi ved godt hvor det ender. Så vi kan altid trykke stop, når det er godt nok. Ingen grund til at træne til varmedøden.



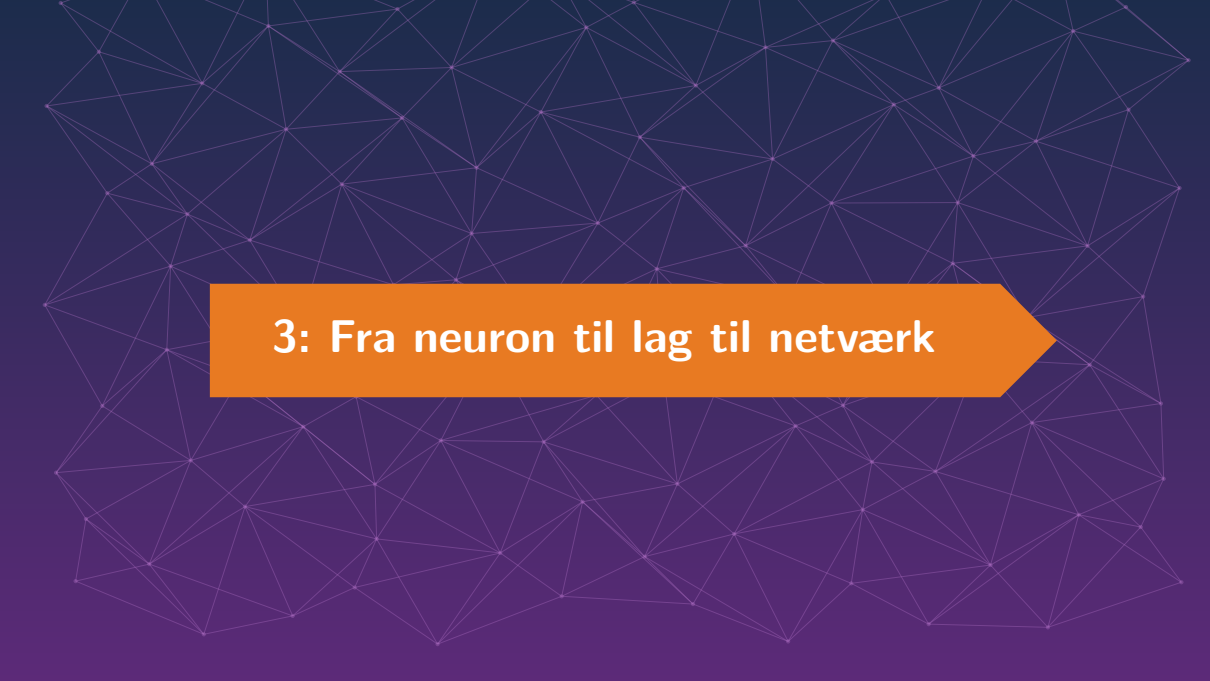
Hvad er en “epoch”?

- Én **epoch** = modellen har set **hele** datasættet én gang.
- Vi kører mange epochs, så vægtene kan justeres en lille smule hver runde.
- I koden: `for epoch in range(400):` betyder “kør 400 runder”.

1 epoch = én runde gennem alle eksempler. 400 epochs = 400 runder.



gennem ALLE eksempler i datasættet



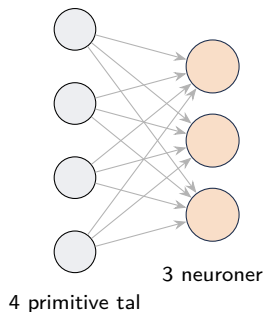
3: Fra neuron til lag til netværk



Et lag = mange neuroner ved siden af hinanden

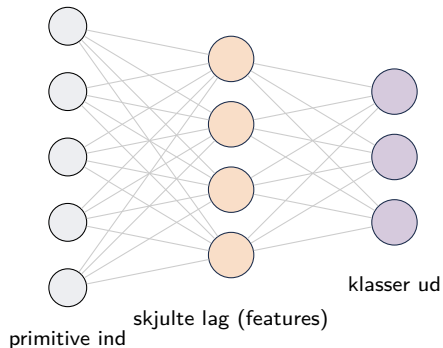
- Ét lag har flere neuroner — hver med sine egne vægte.
- De ser det **samme** input, men leder efter **forskellige** ting.
- I PyTorch: `nn.Linear(4, 3)` = 4 tal ind, 3 neuroner ud.

Flere neuroner = flere spørgsmål stillet på én gang.





Netværk = lag oven på lag



Input → skjulte lag (opdager mønstre) → output (svaret).



Men hvor kommer de features fra?

- Ingen giver netværket “menneske-lignende” eller “stor” direkte.
- Det får kun **primitive** tal, det kan aflæse: **højde, bredde, antal lemmer, farve.**
- Ud af dem bygger de skjulte neuroner *selv* de fundne features.

Primitive tal $\rightarrow$ fundne features $\rightarrow$ klasse. Det er hele netværket.



Fire primitive tal — det man kan SE



Bowser

primitivt træk	værdi
højde	0.95
bredde	0.90
antal lemmer	4
farve	grøn+orange

Højde, bredde og lemmer er tal. Farve bliver også til tal (RGB). Det er hele inputtet.

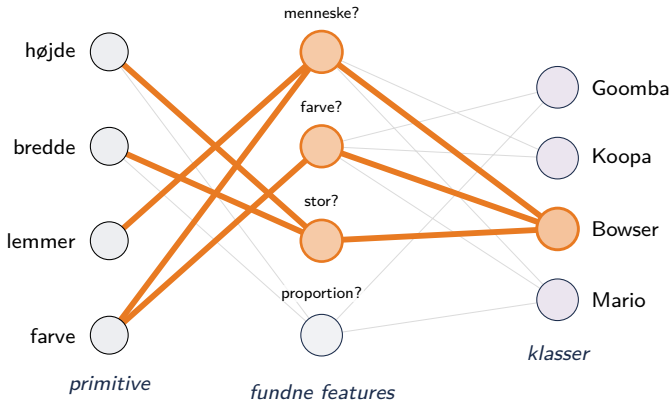


Netværket bygger featuresne selv: Bowser

Nu med **primitive-laget** foran — tykkere streg = stærkere signal.



Bowser



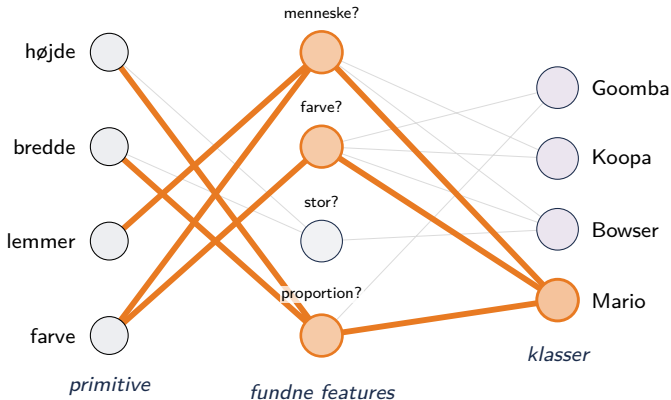
højde+bredde → "stor"; farve+lemmer → "menneske" → Bowser.



Samme net, samme skema: Mario



Mario



Mario tænder farve, menneske og proportioner — men IKKE “stor”.

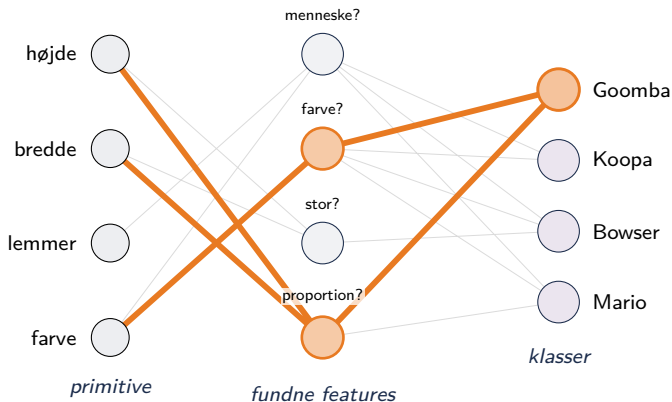
Præcis samme net og samme fire features. Kun *hvilke* der lyser skifter.



...og Goomba — uden “menneske”



Goomba



Goomba: proportioner + farve. Ingen arme → “menneske” forbliver mørk.



Hvad med noget den ALDRIG har set?



Steve fra Minecraft — slet ikke en Mario-figur

- Netværket kender KUN de 4 figurer.
- Det har ingen “det ved jeg ikke”-knap.
- Det aflæser bare højde, bredde, lemmer og farve — og vælger den nærmeste.

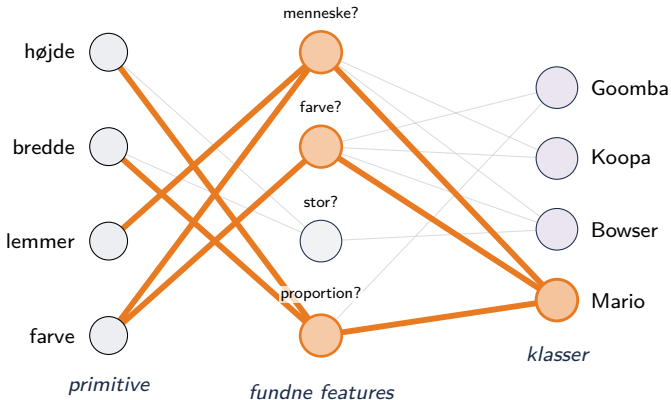
Spørgsmålet er bare: hvilke features kommer til at lyse?



...og især filmens Steve



en fyr i menneske-farver med arme
og ben



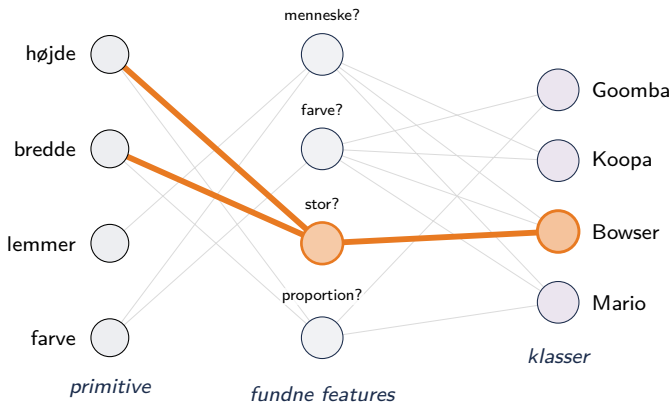
Farve + lemmer → “menneske” lyser → nærmeste klasse er **Mario**.
Han er jo ikke hård eller stor — bare en mand.



En høj Goomba — kør den igennem



en Goomba ... bare strukket
i højden



*Kun "stor" lyser (den blev høj) — og den eneste klasse med "stor" er **Bowser**.*



...men en LANG, flad Goomba?



samme Goomba — bare trukket i bredden

- Farve og proportioner er der stadig nogenlunde.
- Har den lært de *rigtige* features, gætter den stadig “Goomba”.
- Har den kun lært “lille klat”, bliver den nemt snydt.

Gode features generaliserer. Dårlige falder for det første trick.



Derfor er neurale netværk så stærke

- Vi giver rå primitive tal; netværket bygger **sine egne** features i de skjulte lag.
- Jo flere lag, jo mere sammensatte mønstre kan det finde.
- Samme opskrift virker på Pokémon, cifre, billeder og sprog.

Fire primitive tal ind, fire figurer ud — og imellem sidder features, som ingen har programmeret. Ret smart for en bunke gange-stykker.



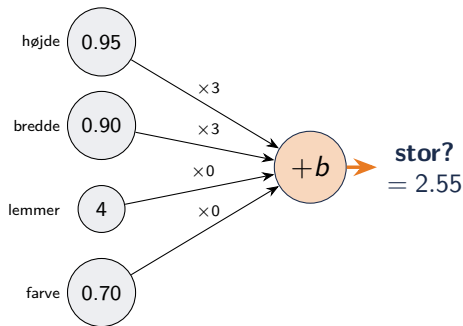
Inde i kassen er det bare tal — én neuron

Hver neuron tager de primitive tal x_1, x_2, x_3, x_4 , ganger hvert med sin vægt og lægger sammen:

$$ud = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b$$

Sæt Bowsers tal ind ($x = [0.95, 0.90, 4, 0.70]$) i en “stor?”-neuron:

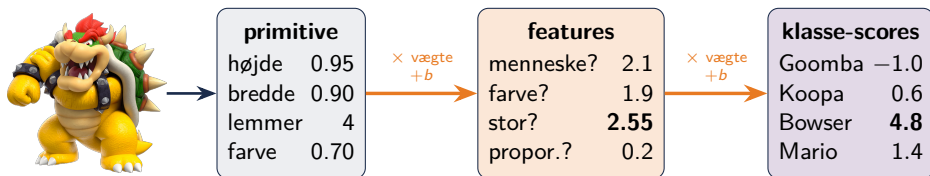
$$= 3 \cdot 0.95 + 3 \cdot 0.90 + 0 \cdot 4 + 0 \cdot 0.70 - 3 = \mathbf{2.55}$$



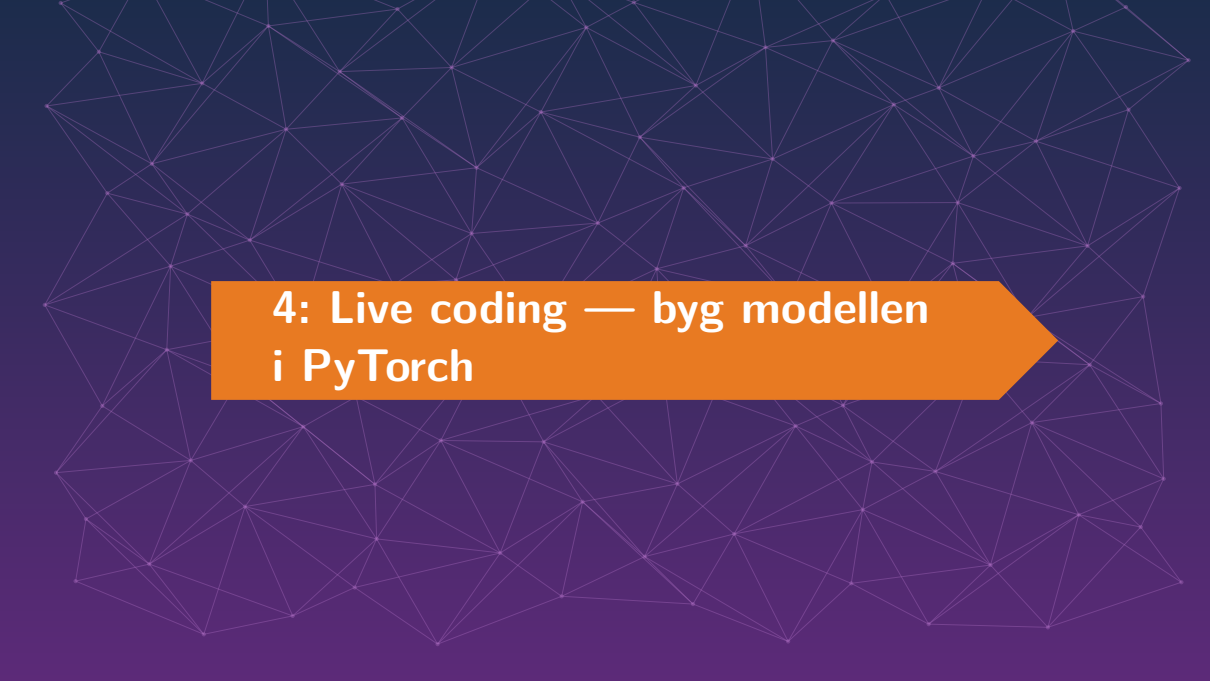
Fire tal ind $\rightarrow$ gang med vægte, læg sammen $\rightarrow$ ét tal ud (her: “stor?” = 2.55).



Samme regnestykke ($w_1x_1 + w_2x_2 + \dots + b$) gentages lag for lag — her for **Bowser**:



*“stor?” = 2.55 (fra før) trækker Bowser-scoren op $\rightarrow$ højeste (4.8) $\rightarrow$ **Bowser**. Kun tal.*



4: Live coding — byg modellen i PyTorch



En model er en klasse (I kender klasser fra intro)

- En PyTorch-model **arver** fra `nn.Module`.
- I `__init__` siger vi *hvilke lag* der er.
- I `forward` siger vi *hvordan data flyder igennem*.

To metoder. Det er hele hemmeligheden.



```
import torch
import torch.nn as nn

class MarioNet(nn.Module):
    def __init__(self):
        super().__init__()          # ALDRIG glem denne linje!
        self.layer1 = nn.Linear(4, 8) # 4 features -> 8 skjulte
        self.activation = nn.ReLU()    # "krølningen" (mere i morgen)
        self.layer2 = nn.Linear(8, 6)  # 8 skjulte -> 6 figurer

    def forward(self, x):              # sådan flyder data igennem
        x = self.activation(self.layer1(x))
        return self.layer2(x)

model = MarioNet()
print(model)
```



Live: den samme trænings-rytme hver gang

```
loss_fn = nn.CrossEntropyLoss()           # "hvor forkert var gættet?"
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

for epoch in range(400):                   # 400 epochs = 400 runder gennem dataen
    optimizer.zero_grad()                  # 1. nulstil
    y_hat = model(X_train)                 # 2. gæet paa alle figurer
    loss = loss_fn(y_hat, y_train)         # 3. hvor forkert?
    loss.backward()                        # 4. find retningen "ned ad bakke"
    optimizer.step()                       # 5. tag et lille skridt
```

Nulstil → gæt → mål fejl → baglæns → skridt. Fem trin. Hver gang.



Åbn notebooken `live-coding/deck1-mario-live.ipynb`

Vi bygger MarioNet sammen på et lille Mario-datasæt, træner den, og kigger på hvad de skjulte neuroner har lært.

- Skru på antal skjulte neuroner — hvad sker der?
- Fjern `self.activation` — pludselig kan den ikke mere (det er i morgens emne).



Nu er det jeres tur

Nu er det jeres tur ✨

Åbn 03-Intro-ML/1-Neural-networks.ipynb og gå i gang med opgaverne.

I bygger et rigtigt netværk der gætter om en **Pokémon er legendarisk** — samme opskrift, andet datasæt.

Hold øje med:

- `super().__init__()` — glemmer man den, brokker PyTorch sig med det samme.
- De fem trin i træningsloopet skal stå i *rigtig* rækkefølge.
- “Den dovne model” der altid gætter det samme — kan I slå den?



Byg jeres første netværk — god fornøjelse!

(og husk: en neuron er bare en linje i festtøj)